Assignment 4: Improving Requirement Reliability in a Multi-Agent Social Media System

Objective

This assignment improves the multi-agent social media system developed and evaluated in the earlier assignments. Assignment 3 showed that a multi-agent pipeline is easier to inspect than a single-agent pipeline, but orchestration alone does not guarantee that user requirements are preserved. If the system extracts the wrong topic or loses a visual constraint, later agents can complete the workflow and still produce an unsuitable post.

The research question for this assignment is:

Can a structured request-verification stage and a requirement-aware QA loop improve the reliability of a multi-agent social media generation pipeline?

Code repository link:

```
https://github.com/tanereskil/agentic_sdlc
```

Part 1 - Baseline System

Problem Definition

The system automates social media post preparation. A user provides a prompt such as “Make a post about Pilates” or a more detailed product request. The pipeline should:

- Infer the post topic and language.
- Decide whether writing, visual generation, QA, and upload are needed.
- Produce a caption.
- Produce a visual brief and image file when media is requested.
- Validate the output.
- Simulate a local browser upload when publishing is requested.

Baseline Agent Roles

The original system for this experiment is the existing full multi-agent pipeline.

Agent	Baseline Role
Task Manager	Creates route, validates steps, retries failed agents
Trend Researcher	Adds keywords and hashtags for short or trend-based prompts
Content Writer	Writes caption and rough media prompt
Visual Prompt Engineer	Converts rough media prompt into structured photo prompt
Media Creator	Creates an image with provider or local placeholder path
QA Agent	Checks caption/media completeness and safety
Browser Operator	Simulates upload to local demo page

Baseline Workflow

```
User Prompt
    |
    v
Task Manager
    |--> Researcher (conditional)
    |--> Writer
    |--> Visual Prompt Engineer (conditional)
    |--> Media Creator (conditional)
    |--> QA Agent
    |--> Browser Operator (conditional)
```

The Task Manager already performs dynamic routing. It can skip research for explicit prompts, skip writer when exact caption text exists, and skip media/upload for text-only drafts.

Current Limitations of the Baseline

The baseline can finish workflows that are technically complete but still weak in requirement alignment. For example:

- A prompt containing an ambiguous reference can produce a generic topic such as `this and it`.
- A content pipeline can approve a post although a specific image instruction is only partially represented.
- Completion success can hide upstream mistakes because every downstream agent works from the initial topic and media prompt.
- More agents produce more external model/API calls in real provider mode, which increases timeout and provider-failure exposure.

These limitations became visible during Assignment 3 experiments and manual screenshots. They define the improvements in this assignment.

Part 2 - Identify Weaknesses

Weakness 1: Low-Confidence Topic Extraction

The baseline Task Manager tries to infer a topic from prompt text. For clear prompts this works. For vague prompts such as “Make a post about this and share it,” the baseline may extract the literal phrase `this and it` as a topic. The remaining agents then receive that topic and produce a generic caption and generic image brief.

Why it occurs:

- The pipeline treats any non-empty topic-like phrase as usable unless it matches a small blocked set.
- The baseline does not maintain an explicit confidence score for topic extraction.
- QA sees a caption and media file, so it can approve the run even though the topic is semantically weak.

Why it matters:

- The system can publish a meaningless post.
- Later agents cannot recover if the initial topic is wrong.
- A completed browser/upload step may incorrectly look like success.

Weakness 2: User Visual Requirements Are Not Explicit Contracts

Some prompts contain visual constraints instead of only a topic. Examples are:

- “Outdoor mat Pilates in a park, not an indoor studio.”
- “A close-up of a woman’s ear with three silver earrings.”

The baseline Writer and Visual Prompt Engineer may preserve many of these details, but the QA Agent mainly checks safety, media existence, and coarse topic alignment. It does not treat every visual condition as an explicit checklist.

Why it occurs:

- Visual constraints are carried in free-form text fields.
- Generic QA approval does not guarantee exact constraint coverage.
- A media prompt can mention the topic `silver earrings` while missing one required detail such as count or close-up framing.

Why it matters:

- A generated post can be plausible but not satisfy the user request.
- Product posts are especially sensitive to count, material, setting, and composition constraints.

Weakness 3: Completion and Cost Do Not Mean Reliability

Multi-agent orchestration adds logs, validation calls, and provider calls. In local deterministic mode this overhead is small. In real API mode, Assignment 3 exposed timeouts and provider connection errors. Extra agent calls increase the surface area for latency and service failures.

Why it occurs:

- Specialized agents create more handoffs than a single combined call.
- Text and image providers can fail independently.
- Baseline success metrics can over-emphasize final completion without separating provider failure from reasoning failure.

Why it matters:

- The system needs verification and fallback behavior, not only more agents.
- Some steps should be stopped early when confidence is low instead of producing an expensive low-quality post.

Part 3 - System Improvements

I implemented three connected improvements. They are designed to improve requirement quality before adding more generation cost.

Improvement 1: Structured Request Brief Verifier

I added `RequestBriefVerifierAgent` in:

```
agents/requirement_brief.py
```

Before the improved Task Manager executes the normal route, this verifier creates a structured brief:

```
{
  "topic": "silver earrings",
  "language": "en",
  "media_required": true,
  "visual_constraints": [
    "close-up composition",
    "earrings visible on a woman's ear",
    "three earrings",
    "silver earrings"
  ],
  "avoid_constraints": [],
  "confidence": 0.92
}
```

The brief separates topic extraction from visual requirement extraction. It also detects ambiguous phrases such as `post about this`.

Improvement 2: Confidence Gate and Human Approval State

The improved pipeline uses `ImprovedTaskManagerAgent` in:

```
agents/improved_task_manager.py
```

If the request brief confidence is low and the topic is ambiguous, the improved system does not proceed blindly. It asks for a topic clarification. Low-confidence runs also mark a human approval requirement before publish. If a pipeline reaches publish with an approval requirement and no approval callback exists, browser upload is blocked with `awaiting_approval`.

This is a controlled human-in-the-loop change:

- High-confidence prompts continue automatically.
- Ambiguous prompts ask for clarification.
- Repaired or low-confidence publish operations can require human approval.

Improvement 3: Requirement-Aware QA and Feedback Repair

I added `RequirementAwareQAAgent` in:

```
agents/requirement_qa.py
```

It runs after normal QA approval and checks the structured request brief:

- Does an outdoor requirement remain visible in the media prompt?
- Does mat Pilates remain a mat-based visual direction?
- Are jewelry count/material/composition requirements present?
- Are forbidden directions such as indoor studio still present in the positive visual prompt?

If this QA rejects a visual requirement mismatch, the improved Task Manager routes the feedback back into the visual branch:

```
Requirement-Aware QA Reject
        |
        v
Task Manager Requirement Repair
        |
        v
Constraint Injection -> Visual Prompt Engineer -> Media Creator -> QA
```

The improved Task Manager also injects request brief visual constraints before visual prompt generation. In the deterministic experiment, this preventive step avoided extra QA repair loops. The feedback loop still exists for

provider/model outputs that violate a constraint after generation.

Implementation Footprint

File	Purpose
`models.py`	Adds request brief, confidence, and human approval state
`agents/requirement_brief.py`	Structured request brief and constraint injection
`agents/requirement_qa.py`	Requirement-aware verification
`agents/improved_task_manager.py`	Improved route, feedback repair, approval gate
`tools/run_assignment4_experiment.py`	Original vs improved experiment runner
`tests/test_assignment4_improvements.py`	Focused improvement tests

Part 4 - Experimental Comparison

Experiment Design

The experiment compares:

- `original_multi_agent`: existing baseline `TaskManagerAgent`
- `improved_multi_agent`: `ImprovedTaskManagerAgent`

Both systems use the same seven prompts. The primary run uses deterministic local mode:

```
/Users/alki/.pyenv/versions/3.13.2/bin/python3 \
  projects/Smart_Social_Media_Agent_Orchestrator/tools/run_assignment4_experiment.py \
  --provider mock \
  --image-provider pillow
```

Output directory:

```
projects/Smart_Social_Media_Agent_Orchestrator/outputs/assignment4_experiment/20260521_145921
```

Test Cases

Case	Weakness Target
A4-T1 Short Pilates	No-regression clear prompt
A4-T2 Ambiguous `this`	Confidence gate and clarification
A4-T3 Outdoor mat Pilates	Preserve environment and mat/studio constraint
A4-T4 Jewelry constraints	Preserve close-up, count, and material
A4-T5 Exact caption	Preserve mixed caption + image intent
A4-T6 Turkish outdoor	Preserve Turkish topic and visual direction
A4-T7 Caption only	Preserve efficient text-only route

Metrics

Metric	Measurement
Overall score	Weighted score for topic, requirements, caption, completeness, QA, media/upload, action
Correctness	Topic, requirement, language, exact-caption, and QA outcome
Requirement score	Coverage of expected visual requirement groups and forbidden visual terms
Clarification score	For ambiguous prompt, whether system asks for topic instead of publishing
Completeness	Caption, required media, required upload, and QA status
Failure rate	Wrong expected action, exception, or non-approved QA for complete-output cases
Latency	Wall-clock milliseconds
LLM step proxy	Model-capable planned phases
Logs and approvals	Coordination overhead and human gate evidence

Aggregate Results

System	Mean Overall	Correctness	Requirement	Completeness	Latency ms	Logs	LLM Step Proxy	Human Approvals	Failure Rate
Original Multi-Agent	0.85	0.85	0.95	0.86	15.02	13.71	3.00	0.00	0.14
Improved Multi-Agent	1.00	1.00	1.00	1.00	10.42	14.71	2.43	0.14	0.00

The improved mean latency is lower in this deterministic table mainly because the improved system stops early on the ambiguous A4-T2 prompt instead of generating a full low-confidence post. For complete-output cases, the improved workflow usually has extra log entries because it adds verification.

Key Per-Test Results

Case	Original Result	Improved Result	Interpretation
A4-T2 Ambiguous `this`	Published with topic `this and it`; score 0.00	Asked for topic clarification; score 1.00	Confidence gate prevents meaningless publish
A4-T4 Jewelry constraints	Requirement score 0.67	Requirement score 1.00	Structured constraints improve image brief coverage
A4-T1, T3, T5, T6, T7	Complete and approved	Complete and approved	Improvements did not break normal routes

Representative Logs

Original ambiguous flow:

```
topic: this and it
qa_status: approved
browser_status: success
```

Improved ambiguous flow:

```
request_brief_verifier low_confidence
task_manager clarification_needed
question: What specific topic or product should the post cover?
```

Improved constraint-aware flow:

```
request_brief_verifier constraints_applied
visual_prompt_engineer validated
requirement_qa approved
```

Part 5 - Discussion

Which Improvements Worked Well?

The structured request brief worked well. It created a clear topic/constraint boundary before generation. The largest gain appeared in the ambiguous prompt: the original system completed a meaningless post, while the improved system treated the same input as insufficient information.

Constraint injection and requirement-aware QA also improved product visual reliability. In the jewelry case, the original result was approved but received only `0.67` visual requirement coverage. The improved result preserved the close-up, material, and count expectations and reached `1.00`.

Which Improvements Did Not Add Visible Benefit in Every Case?

The improved system adds extra logs and verifier state even for easy prompts. For T1, T3, T5, T6, and T7 both systems already produced correct deterministic outputs. The extra verification did not change the final score there.

The requirement repair feedback loop did not trigger in the deterministic aggregate run. This is not a bug: the pre-generation request brief constraint injection prevented the tested mismatches before QA. The repair path is still needed for real provider outputs, where a text or image model may ignore instructions after prompt generation.

Unexpected Outcomes

Mean latency became lower for the improved system in the deterministic run. This does not mean verification is always faster. It occurs because the improved ambiguous run stops after three logs, while the original baseline continues through writing, media generation, QA, and browser upload. The result supports early stopping as an efficiency strategy for low-confidence requests.

New Problems or Trade-Offs

The improvements introduce new complexity:

- More state fields must remain consistent.
- The verifier rules need maintenance as new domains and languages are added.
- Human approval gates can slow publishing if triggered too often.
- Requirement scoring currently checks prompt coverage, not the final aesthetics of a generated image.

The improved design is still better for this workflow because it makes uncertain decisions explicit rather than hiding them behind a completed pipeline.

Part 6 - Reflection: Human + AI Collaboration

Humans should not manually approve every low-risk draft. AI agents can independently route work, draft captions, generate image prompts, validate file existence, and prepare local previews when confidence is high and requirements are clear.

Human oversight is appropriate when:

- Topic confidence is low.
- A post is being published to a real external account.
- Brand, legal, safety, or sensitive product claims are involved.
- QA had to repair a requirement mismatch.
- Generated media must accurately represent a product or person.

The practical design principle is selective oversight. Humans should remain in the loop for high-impact or low-confidence decisions, while agents handle repetitive drafting and validation steps. A reliable agentic system is not one that runs autonomously at all costs. It is one that knows when autonomy is justified and when uncertainty should be exposed to a human.

Responsible Use of AI

AI tools were used to help implement, test, and document the system. I remain responsible for the experiment design, metric definitions, interpretation of results, and limitations. The deterministic experiment is intentionally separated from real provider behavior so that architectural improvements are not confused with temporary API availability.